

Informe del módulo de evaluación

CSDI RAG Engine

1 Módulo de evaluación

El módulo de evaluación es el componente encargado de medir la calidad de los rankings producidos por el sistema de recuperación de información. Su responsabilidad no es recuperar documentos ni generar respuestas, sino construir un marco reproducible para comparar estrategias de recuperación usando consultas, rankings, juicios de relevancia y métricas estándar de Information Retrieval. En el proyecto, este módulo permite evaluar de forma separada los métodos BM25, vectorial e híbrido, y justificar empíricamente cuál de ellos entrega mejores resultados sobre el corpus indexado.

La motivación del módulo surge de una necesidad concreta: un sistema RAG puede generar respuestas aparentemente correctas, pero la calidad de la respuesta depende en gran medida de los fragmentos que se recuperan antes de llamar al modelo de lenguaje. Por tanto, no basta con observar la respuesta final; es necesario evaluar si el recuperador coloca documentos relevantes en las primeras posiciones. El módulo de evaluación introduce esa capa de validación y permite analizar la recuperación como un problema independiente de la generación.

Rol dentro de la arquitectura general. El flujo de evaluación se conecta con los recuperadores existentes del sistema. Para una consulta de evaluación, el backend puede generar rankings usando BM25, búsqueda vectorial o recuperación híbrida. Cada ranking es una lista ordenada de fragmentos identificados por `chunk_id`. Posteriormente, esos fragmentos se comparan contra un conjunto de juicios de relevancia, conocido como *qrels*, para calcular métricas cuantitativas.

Esta separación mantiene el módulo aislado del resto de la arquitectura. El evaluador no modifica el índice, no cambia la lógica de recuperación y no interviene en el pipeline RAG. Solo consume los rankings generados y los juicios disponibles. Esto permite usarlo tanto en modo offline, mediante archivos JSON versionados, como desde el frontend, donde un profesor puede crear consultas, generar rankings, asignar relevancias y ejecutar el cálculo de métricas.

Estructura del dataset de evaluación. El dataset base se organiza en tres archivos principales. El archivo `queries.json` contiene las consultas de evaluación, cada una con un identificador estable, el texto de la consulta y, opcionalmente, el alcance de fuentes sobre el que debe evaluarse. El archivo `rankings.generated.json` almacena los rankings generados para cada consulta y estrategia. El archivo `qrels.json` contiene los juicios de relevancia manuales para cada par `query_id + chunk_id`.

El dataset inicial incluye cinco consultas sobre documentación de Python. Para cada una se generaron rankings con $k = 10$ usando BM25, recuperación vectorial e híbrida. La unión de esos rankings produjo 150 posiciones evaluables, pero solo 98 pares únicos consulta-fragmento debido a que varios fragmentos aparecen en más de una estrategia. Todos esos pares fueron juzgados manualmente con una escala de relevancia graduada de 0 a 3.

La escala usada es la siguiente: 0 significa no relevante, 1 indica relevancia marginal, 2 indica relevante y 3 indica muy relevante. Para las métricas binarias, como Precision, Recall, F1 y MRR, el sistema considera relevante cualquier fragmento con valor mayor o igual que 2. Para NDCG se

conserva la escala graduada completa, porque esta métrica sí distingue entre documentos relevantes y muy relevantes.

Auditoría de completitud. Una parte importante del diseño es la auditoría del dataset. El módulo incluye una utilidad que compara los rankings generados contra los qrels y verifica si todos los fragmentos recuperados tienen juicio de relevancia. Esta verificación evita calcular métricas sobre un conjunto incompleto de juicios, situación que puede favorecer artificialmente a una estrategia si solo se juzgaron los documentos que ella recuperó.

El estado final del dataset base confirma su completitud: existen 5 consultas, 150 posiciones de ranking, 98 pares únicos consulta-fragmento, 98 pares juzgados y 0 pares pendientes. Esto significa que todos los resultados actualmente recuperados por BM25, vector e híbrido para las cinco consultas base tienen una relevancia asignada. Esta condición hace que el reporte final sea reproducible y defendible.

Generación de rankings. La generación de rankings puede ejecutarse desde la API o mediante utilidades offline. Para cada consulta, el backend invoca la estrategia seleccionada y obtiene los primeros k resultados. Si la consulta define un filtro de fuentes, el sistema solicita una cantidad mayor de candidatos, enriquece los resultados con metadatos, filtra por `source_id` y finalmente conserva los primeros k documentos válidos. Esta decisión evita que una consulta restringida a una fuente devuelva menos resultados simplemente porque el filtro se aplicó después de recuperar solo k candidatos globales.

El campo `source_ids` permite dos comportamientos. Si su valor es nulo, la evaluación se realiza contra todo el corpus indexado. Si contiene una lista de fuentes, la evaluación se limita a esas fuentes. Así, una misma infraestructura permite datasets generales, datasets por fuente y datasets mixtos. Esta flexibilidad es útil para un sistema que puede crecer incorporando documentación de Python, JavaScript u otras tecnologías.

Métricas implementadas. El módulo implementa cinco métricas principales. Precision@K mide qué proporción de los primeros K resultados recuperados son relevantes:

$$Precision@K = \frac{|Rel_K|}{K}$$

donde Rel_K es el conjunto de documentos relevantes dentro del top K . Esta métrica es útil cuando importa la calidad inmediata de los primeros resultados.

Recall@K mide qué proporción de todos los documentos relevantes conocidos aparece dentro del top K :

$$Recall@K = \frac{|Rel_K|}{|Rel|}$$

donde Rel es el conjunto total de documentos juzgados como relevantes para la consulta. Esta métrica penaliza a una estrategia cuando deja fuera documentos relevantes conocidos.

F1@K combina Precision y Recall mediante media armónica:

$$F1@K = \frac{2 \cdot Precision@K \cdot Recall@K}{Precision@K + Recall@K}$$

Su función es ofrecer una medida balanceada entre calidad de los primeros resultados y cobertura de documentos relevantes.

MRR mide qué tan temprano aparece el primer documento relevante. Para una consulta, se calcula como el inverso de la posición del primer resultado relevante:

$$RR = \frac{1}{rank_{first\ relevant}}$$

En el reporte agregado se promedia sobre todas las consultas. Un valor de 1 indica que el primer resultado del ranking ya es relevante.

NDCG@K evalúa la calidad del orden considerando relevancia graduada. Primero se calcula el Discounted Cumulative Gain:

$$DCG@K = \sum_{i=1}^K \frac{2^{rel_i} - 1}{\log_2(i + 1)}$$

Luego se normaliza dividiendo por el DCG ideal, que corresponde al orden perfecto de los documentos juzgados:

$$NDCG@K = \frac{DCG@K}{IDCG@K}$$

Esta métrica es especialmente adecuada para el módulo porque los juicios usan valores 0, 1, 2 y 3. No solo importa recuperar documentos relevantes, sino colocarlos en el orden correcto y priorizar los más útiles.

Flujo operativo desde la API y el frontend. El backend expone endpoints para listar y crear consultas, generar rankings, consultar rankings persistidos, actualizar juicios de relevancia, ejecutar la evaluación y leer el reporte final. Sobre esos endpoints, el frontend incorpora una sección de evaluación dentro del explorador de búsqueda. Desde allí el profesor puede seleccionar una consulta base, elegir el alcance de fuentes, generar rankings para BM25, vector e híbrido, asignar relevancias y ejecutar el cálculo de métricas.

La interfaz separa la exploración normal de la evaluación. En exploración, el usuario prueba búsquedas libres. En evaluación, el usuario construye o revisa un dataset. Además, existe una sección didáctica de métricas que explica Precision@K, Recall@K, F1@K, MRR y NDCG@K. Esta decisión ayuda a que el módulo no sea solo una herramienta técnica, sino también un recurso comprensible para analizar resultados durante la defensa del proyecto.

Resultados obtenidos. El reporte final compara las tres estrategias sobre las cinco consultas base con $K = 10$. Los valores promedio son:

Estrategia	Precision@10	Recall@10	F1@10	MRR	NDCG@10
BM25	0.460	0.520	0.487	0.840	0.543
Vector	0.540	0.647	0.586	0.850	0.670
Híbrido	0.620	0.715	0.662	1.000	0.759

La estrategia híbrida obtiene el mejor promedio en todas las métricas principales. Esto confirma la hipótesis de diseño del sistema: combinar señales léxicas y semánticas produce rankings más robustos que usar una sola estrategia. BM25 conserva valor para términos exactos, mientras que la recuperación vectorial mejora la cobertura semántica. La fusión híbrida aprovecha ambas señales y logra mejores posiciones para los documentos relevantes.

Persistencia y reproducibilidad. La reproducibilidad se logra versionando consultas, qrels, rankings generados, reporte final y reporte de auditoría. Esto permite recalcular métricas, revisar juicios de relevancia y verificar que el dataset no tenga pares pendientes. El módulo también incluye pruebas unitarias para las métricas, el evaluador, el cargador de datasets, el runner offline, el recolector de rankings, el store JSON y la auditoría de qrels.

Esta combinación de archivos y pruebas permite defender el módulo como un componente verificable. No depende de observaciones manuales aisladas ni de una ejecución puntual del frontend. El estado del dataset puede auditarse y el reporte puede regenerarse siguiendo los comandos documentados en el README del módulo.

Limitaciones y mejoras futuras. La principal limitación del módulo es el tamaño del dataset base. Cinco consultas permiten demostrar el flujo completo y comparar estrategias, pero no bastan para caracterizar de forma exhaustiva todo el comportamiento del sistema. Para una evaluación más sólida, el dataset debería ampliarse con consultas de distintas tecnologías, diferentes niveles de dificultad, preguntas conceptuales, búsquedas por errores, consultas de API y casos donde la respuesta requiera varios fragmentos.

Otra limitación es que los juicios de relevancia fueron asignados manualmente por un evaluador. Este enfoque es válido para un dataset académico pequeño, pero puede introducir sesgos. Una mejora futura sería permitir múltiples evaluadores, calcular acuerdo entre anotadores y resolver discrepancias. También sería útil agregar exportación de datasets, filtros para resultados pendientes y métricas por fuente o por tipo de consulta.

Conclusión. El módulo de evaluación incorpora una capa formal de medición sobre el sistema de recuperación. Define consultas, rankings, qrels, métricas, auditoría y reporte final. Además, expone una API operable desde el frontend y mantiene los artefactos necesarios para reproducir los resultados. Con el dataset base completo y `missing_pairs = 0`, el sistema puede comparar BM25, recuperación vectorial e híbrida de manera clara, cuantitativa y defendible. Los resultados muestran que la estrategia híbrida ofrece el mejor desempeño promedio, lo que respalda la arquitectura de recuperación combinada usada por el CSDI RAG Engine.